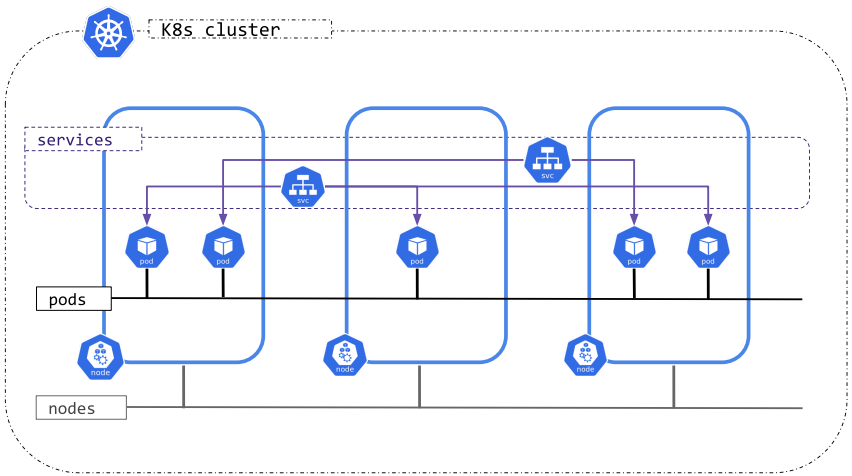
ITBA

Arquitectura de Microservicios con Contenedores

Docentes:

- Ing. Federico Castañeda
- Ing. Carlos Maceira
- Ing. Franco Rupnik

Networking • Rangos de IP del clúster



Fuente: Kubernetes – Cluster Networking

- **Nodos:** IPs de las interfaces sobre la red de infraestructura (*underlay*).
- **Pods:** rango asignado por el plugin de red; cada Pod tiene IP y puede comunicarse con los demás.
- **Services:** rango virtual; una **ClusterIP** ofrece un destino estable y reenvía a los Pods.

El **CNI** decide cómo viajan los paquetes Pod-Pod: por rutas de la red de nodos o encapsulados dentro de ella (*overlay*). La overlay es una implementación posible, no un requisito de Kubernetes.

Networking

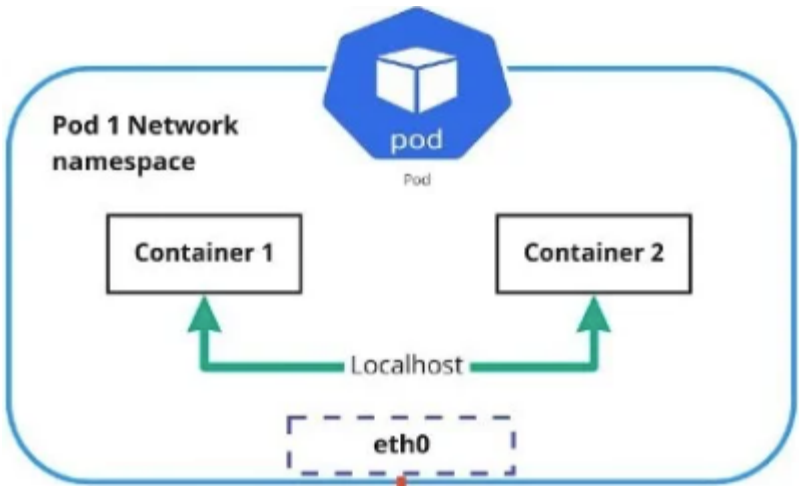
Hay cuatro necesidades de comunicación dentro de K8S:

- Entre contenedores que están altamente acoplados, ejecutándose dentro del mismo pod.
- De un Pod a otro Pod.
- De un Pod a un Servicio
- Externamente hacia un Servicio

Networking • Dentro del Pod

Un Pod agrupa **contenedores** que comparten el mismo namespace de red.

- Comparten sus direcciones IP y puertos; se comunican por **localhost**.
- Todos los contenedores del Pod se ejecutan en el mismo nodo.
- Pods distintos no comparten localhost: cada uno tiene su propio namespace de red.
- Las IPs pueden cambiar al recrear el Pod.

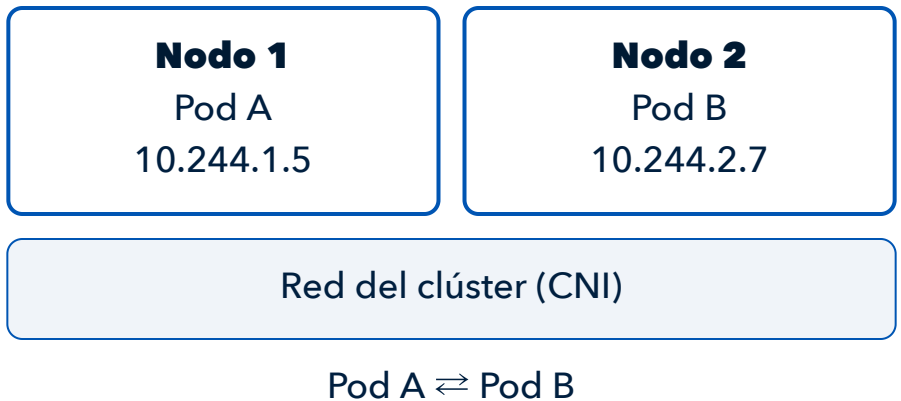


Networking • Entre Pods

La **red del clúster** conecta los Pods, tanto en el mismo nodo como entre nodos.

- Cada Pod recibe IPs únicas: IPv4, IPv6 o dual-stack.
- **CNI** (*Container Network Interface*) es el estándar por el que el runtime invoca un plugin de red. El plugin configura la conexión del Pod y le asigna sus IPs; el modelo de Kubernetes permite comunicación Pod–Pod sin NAT.
- La topología depende de la implementación: no exige una subred dedicada por nodo.
- **kube-proxy** (o una alternativa) configura el reenvío de Services hacia sus Pods; no conecta Pods entre sí.

Para no depender de IPs efímeras, se utiliza un Service.



Networking · Overlay o ruteo nativo

El CNI elige cómo transportar el paquete Pod-Pod entre nodos.

Ruteo nativo

Pod A `10.244.1.5` → Pod B `10.244.2.7`

La red conoce la ruta `10.244.2.0/24 → Nodo B`. El paquete conserva las IPs de los Pods.

Overlay (VXLAN)

Nodo A `192.168.1.10` → Nodo B `192.168.1.11`

Dentro viaja el paquete Pod A `10.244.1.5` → Pod B `10.244.2.7`.

Mantenibilidad

El underlay no necesita rutas de Pods; hay que operar túneles y MTU.

Extensibilidad

Funciona sobre redes que desconocen las IPs de Pods; acceder a esas IPs desde afuera suele requerir un gateway.

Seguridad

Suele mantener las IPs de Pods fuera del ruteo del underlay. No cifra ni reemplaza NetworkPolicy.

Performance

La encapsulación agrega encabezados y trabajo; el ruteo nativo suele ser más eficiente.

Fuentes: Calico · Cilium · NetworkPolicy

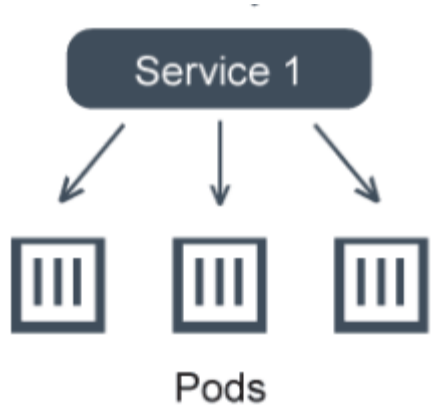
Networking • Pods a un Servicio

Un **Service** ofrece un acceso estable a uno o más Pods, sin depender de sus IPs ni de su ciclo de vida. Su **ClusterIP** es una IP virtual interna que representa ese acceso.

`kube-proxy` o una implementación alternativa programa el reenvío desde esa IP virtual hacia los Pods.

El Service ofrece un destino estable, no permisos de acceso. **NetworkPolicy** restringe el tráfico si el plugin de red aplica esas reglas.

El balanceo depende de la implementación; no garantiza round-robin ni reparto por solicitud HTTP.

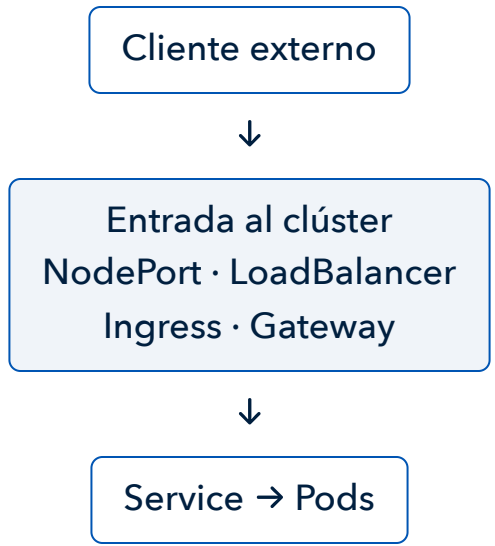


Networking • Externamente a un servicio

La ClusterIP se utiliza normalmente dentro del clúster. Para publicar el servicio hacia afuera existen distintas opciones:

- **NodePort / LoadBalancer:** exponen puertos de transporte.
- **Ingress:** enruta HTTP/S por host y path.
- **Gateway API:** alternativa recomendada para nuevos despliegues; admite enrutamiento L4/L7 según sus recursos y la implementación.
- **kubectl port-forward:** acceso temporal para desarrollo y diagnóstico.

Ingress y **Gateway** declaran reglas de entrada; un controlador debe implementarlas para que el tráfico llegue al Service.



EndpointSlice

Un EndpointSlice lista **direcciones y puertos** de los backends de un Service.

- API **`discovery.k8s.io/v1`**, estable desde v1.21.
- Agrupa endpoints por Service, familia IP y puertos; por defecto, hasta **100 por slice**.
- Soporta **IPv4/IPv6**, readiness y topología.
- El controlador actualiza los slices al cambiar los Pods o su disponibilidad.

La API **Endpoints está deprecada desde v1.33**: no soporta dual-stack y trunca listas de más de 1000 endpoints.

Ejemplo de un slice generado:

```yaml
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  name: my-nginx-svc-abc
  labels:
    kubernetes.io/service-name: my-nginx-svc
addressType: IPv4
ports:
- protocol: TCP
  port: 80
endpoints:
- addresses: ["10.244.0.10"]
  conditions: { ready: true }
- addresses: ["10.244.0.11"]
  conditions: { ready: true }
- addresses: ["10.244.0.12"]
  conditions: { ready: true }
```

Services

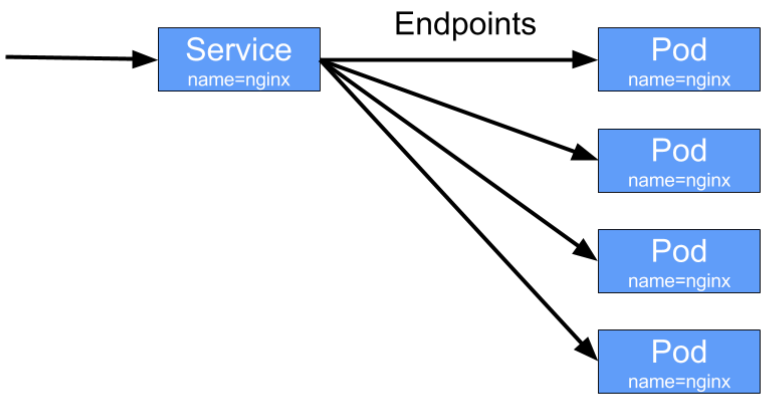
El **selector** determina qué Pods integran un Service. Kubernetes mantiene sus direcciones y puertos en **EndpointSlices**.

El cliente usa el nombre DNS –que normalmente resuelve a la ClusterIP– o esa IP virtual, sin conocer las IPs de los Pods.

```
mi-svc.mi-ns.svc.cluster.local
```

No todos los Services tienen IP virtual:

- **Headless:** DNS devuelve las IPs de los Pods.
- **ExternalName:** DNS devuelve un CNAME externo, sin proxy.



Services • Deployment y Pods

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.30.5
        ports:
        - containerPort: 80
```

Salida ilustrativa de `kubectl get pods`:

```
nginx-deployment-<hash>-klqjc  1/1  Running
nginx-deployment-<hash>-m4r9g  1/1  Running
nginx-deployment-<hash>-w4mnq  1/1  Running
```

IPs ilustrativas de los Pods:

```
10.244.0.10  minikube
10.244.0.11  minikube
10.244.0.12  minikube
```

Services • Service y EndpointSlice

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-nginx-svc
spec:
  selector:
    app: nginx
  ports:
  - protocol: TCP
    port: 80
```

El controlador deriva los EndpointSlices del selector y del estado de los Pods.

```bash
kubectl get endpointslices \
  -l kubernetes.io/service-name=my-nginx-svc \
  -o yaml
```

Fragmento ilustrativo de la salida:

```yaml
# Dentro de items[0]
addressType: IPv4
ports:
- protocol: TCP
  port: 80
endpoints:
- addresses: ["10.244.0.10"]
- addresses: ["10.244.0.11"]
- addresses: ["10.244.0.12"]
```

Service • Acceso a un Pod

Acceso a un Pod por medio de la IP del servicio

```
/# wget -O - 10.96.36.226
Connecting to 10.96.36.226 (10.96.36.226:80)
writing to stdout
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

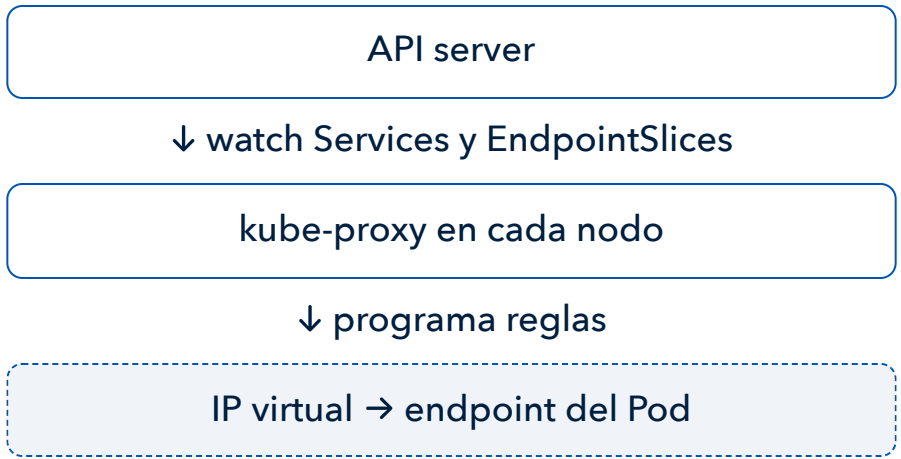
Acceso a un Pod por medio de DNS service.namespace

```
/# wget -O - my-nginx-svc.default
Connecting to my-nginx-svc.default (10.96.36.226:80)
writing to stdout
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
```

Kube-Proxy

kube-proxy programa el plano de datos de los Services en cada nodo:

- Observa **Services y EndpointSlices** mediante el **API server**.
- Configura reglas para redirigir tráfico de `ClusterIP:puerto` hacia un endpoint disponible.
- Implementa IP virtual, NAT y balanceo; no la conectividad base Pod-Pod.
- No hay un proxy central: el reenvío se resuelve en los nodos.



Kube-Proxy • Modos y alternativas

Modos en Linux

- **iptables:** vigente y predeterminado en Kubernetes 1.37.
- **nftables:** alternativa estable; requiere kernel 5.13 o superior.
- **IPVS:** deprecado desde v1.35.

En Windows se utiliza **kernelspace**.

Otras implementaciones

Algunas redes pueden sustituir kube-proxy por su propio mecanismo de acceso a Services.

Por ejemplo, **Cilium con kube-proxy replacement** utiliza eBPF para el balanceo de Services.

La función se mantiene: encaminar el acceso estable del Service hacia sus backends actuales.

Cluster DNS

El DNS del clúster (habitualmente CoreDNS) permite descubrir Services por nombre: resuelve a la ClusterIP, a las IPs de los Pods en un Service headless o a un CNAME en un ExternalName.

```
/ # cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5

/ # nslookup my-nginx-svc.default.svc.cluster.local
Server:     10.96.0.10
Address:    10.96.0.10:53

Name:       my-nginx-svc.default.svc.cluster.local
Address:    10.96.36.226
```

ClusterIP vs Headless

ClusterIP

- IP virtual estable delante de los Pods.
- DNS resuelve a la ClusterIP del Service.
- El selector determina los backends.
- kube-proxy o una alternativa programa el reenvío.
- No garantiza round-robin: iptables/nftables seleccionan por conexión, no por solicitud HTTP.

Headless

- `clusterIP: None`: **sin IP virtual ni proxy de Service**.
- Con selector, DNS devuelve las IPs de los Pods elegibles (A/AAAA).
- El **cliente elige** a qué Pod conectarse.
- Permite descubrir réplicas directamente o implementar balanceo en la aplicación.

ClusterIP vs Headless • YAML

ClusterIP

```yaml
apiVersion: v1
kind: Service
metadata:
  name: regular-service
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Headless

```yaml
apiVersion: v1
kind: Service
metadata:
  name: headless-svc
spec:
  clusterIP: None
  selector:
    app: web
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
```

Tipos de Servicios

ClusterIP

IP interna para acceder al servicio desde el clúster.

NodePort

- Publica un puerto en los nodos y conserva una ClusterIP.
- Rango **predeterminado** 30000–32767, configurable.
- Su accesibilidad depende de las IPs habilitadas, la red y los firewalls.

LoadBalancer

- Requiere integración con un proveedor o controlador; no exige cloud.
- Normalmente asigna NodePorts; **no prohíbe** el acceso directo a ellos.
- Puede omitirlos con `allocateLoadBalancerNodePorts: false` si la implementación lo soporta.

ExternalName

CNAME hacia un nombre externo, sin proxy ni IP virtual.

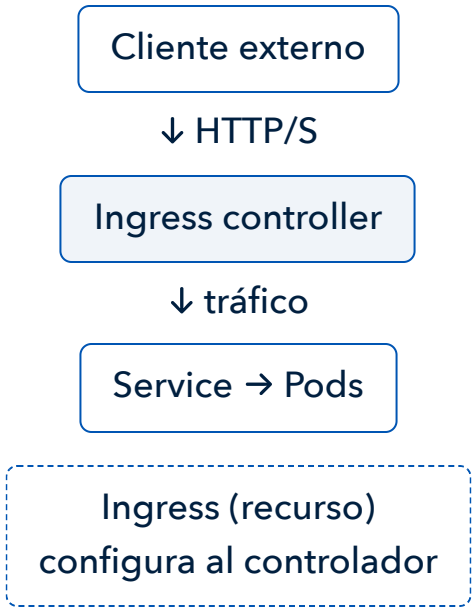
Tipos de Servicios • kubectl

```
[k8s-services]$ kubectl get svc
NAME                        TYPE           CLUSTER-IP       EXTERNAL-IP     PORT(S)
kubernetes                  ClusterIP      10.128.0.1       <none>          443/TCP
mongodb-service             ClusterIP      10.128.204.105   <none>          27017/TCP
mongodb-service-headless    ClusterIP      None             <none>          27017/TCP
ms-service-loadbalancer     LoadBalancer   10.128.233.22    172.104.255.5   3200:30010/TCP
ms-service-nodeport         NodePort       10.128.202.9     <none>          3200:30008/TCP
```

Ingress y su controlador

Un **Ingress** declara rutas HTTP/S por **host y path** hacia Services internos.

- Requiere un **controlador instalado** que implemente las reglas.
- API `networking.k8s.io/v1`: estable desde v1.19 y actualmente **congelada**, no deprecada ni en retirada.
- **ingress-nginx comunitario** fue retirado en marzo de 2026, sin nuevas correcciones de seguridad.
- No confundirlo con otros controladores NGINX.
- Para nuevos despliegues se recomienda **Gateway API**.



Ingress • Reglas y clase

`ingressClassName` referencia una **IngressClass** asociada al controlador.

Las reglas combinan host, path, `pathType` y backend (Service y puerto).

La clase y el controlador de este ejemplo son **ilustrativos**: deben reemplazarse por una implementación instalada y mantenida.

```yaml
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: example-class
spec:
  controller: example.com/ingress-controller
```

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-nginx
  namespace: default
spec:
  ingressClassName: example-class
  rules:
  - host: nginx.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: my-nginx-svc
            port:
              number: 80
```

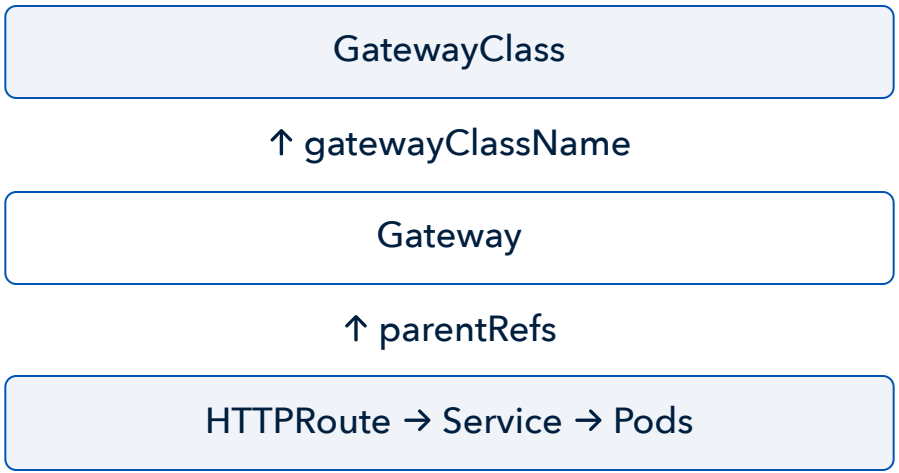
Gateway API • Modelo de recursos

Gateway API amplía el modelo de entrada con recursos **L4/L7** y separación de responsabilidades:

- **GatewayClass:** tipo de gateway y controlador.
- **Gateway:** infraestructura de entrada y listeners.
- **HTTPRoute:** reglas HTTP y sus backends.

El operador administra la infraestructura; las aplicaciones declaran rutas.

Requiere **CRDs e implementación compatible**, instalados aparte. Los objetos solos no habilitan tráfico.



Gateway API • Ejemplo

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: example-gateway-class
spec:
  controllerName: example.com/gateway-controller
```

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
  namespace: default
spec:
  gatewayClassName: example-gateway-class
  listeners:
  - name: http
    protocol: HTTP
    port: 80
```

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-nginx-route
  namespace: default
spec:
  parentRefs:
  - name: my-gateway
  hostnames:
  - nginx.example.com
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /
    backendRefs:
    - name: my-nginx-svc
      port: 80
```

Clase y controlador **ilustrativos**: requieren una implementación instalada.

NetworkPolicy

Publicar una ruta no implica permitir cualquier comunicación. **NetworkPolicy** restringe el tráfico de los Pods.

- API `networking.k8s.io/v1`, reglas **L3/L4**.
- `podSelector` selecciona Pods del namespace.
- `policyTypes` indica **Ingress**, **Egress** o ambas.
- Sin políticas que lo seleccionen en una dirección, el Pod no está aislado por NetworkPolicy en esa dirección.

Reglas de aislamiento

- Las políticas son **aditivas**: se permite la unión de sus reglas, sin prioridad por orden.
- Si ambos extremos están aislados, deben permitir la conexión la **egress del origen** y la **ingress del destino**.
- Requiere una **implementación de red compatible**; sin enforcement, el objeto no tiene efecto.

No autentica ni cifra, y no filtra HTTP por URL.

NetworkPolicy • Default deny + allow

Aislar el tráfico entrante de todos los Pods de `default`:

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: default
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Otras políticas pueden habilitar conexiones específicas. Esta no aísla el tráfico saliente.

El acceso desde el Gateway requiere una regla que permita su origen.

Permitir `app=frontend` del **mismo namespace** hacia `app=nginx`, puerto 80:

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-ingress
  namespace: default
spec:
  podSelector:
    matchLabels: { app: nginx }
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels: { app: frontend }
    ports:
    - protocol: TCP
      port: 80
```

Storage

Storage en Kubernetes

Kubernetes no proporciona por sí mismo un backend persistente. El ciclo de vida depende de dónde se almacenen los datos:

Almacenamiento	Ciclo de vida	Qué sobrevive
Capa escribible del contenedor	Contenedor	No sobrevive al reinicio o reemplazo del contenedor
`emptyDir`	Pod	Reinicios del contenedor, no la eliminación del Pod
Volumen persistente (PV/PVC)	Independiente del Pod	Reemplazo del Pod, sujeto al backend y a la retención

No todos los volúmenes son efímeros. Un PV permite separar los datos del ciclo de vida del Pod.

Storage • Garantías y límites

Disponibilidad y topología

No es necesario que un volumen sea accesible desde todos los nodos.

- Un disco zonal está disponible en su zona.
- Un volumen local está ligado a un nodo.
- El scheduling debe respetar esas restricciones, por ejemplo mediante `nodeAffinity`.

Durabilidad

Sobrevivir a la destrucción del clúster **no es una garantía de Kubernetes**.

El PV es un objeto de Kubernetes; los datos viven en el backend.

Su conservación depende de la infraestructura y de la política de retención. Persistencia no significa disponibilidad ilimitada.

PersistentVolume

Un **PersistentVolume (PV)** representa un recurso de almacenamiento: disco local, volumen de red u otro backend.

Su ciclo de vida es independiente de los Pods que lo consumen. Se describe mediante:

- **capacity:** capacidad ofrecida.
- **accessModes:** modos de acceso soportados.
- **persistentVolumeReclaimPolicy:** qué sucede con el almacenamiento cuando se elimina su claim.

PersistentVolume • Acceso y retención

Modos de acceso

- **RWO / ReadWriteOnce:** lectura/escritura desde un nodo; puede haber varios Pods en él.
- **ROX / ReadOnlyMany:** lectura desde varios nodos.
- **RWX / ReadWriteMany:** lectura/escritura desde varios nodos.
- **RWOP / ReadWriteOncePod:** un único Pod; requiere CSI, estable desde v1.29.

No son permisos de acceso ni sustituyen la protección de escritura.

Reclaim policy

Al eliminarse el PVC:

- **Retain:** conserva el volumen y sus datos; recuperación manual.
- **Delete:** elimina el PV y el almacenamiento subyacente, si el backend lo soporta.
- **Recycle:** deprecado; no se recomienda.

Los PV dinámicos heredan la política de su StorageClass; el valor predeterminado es **Delete**.

PersistentVolume (NFS)

Ejemplo **estático**: el administrador crea el PV. NFS sigue vigente; requiere un servidor disponible con el recurso exportado.

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs
spec:
  capacity:
    storage: 1Mi
  accessModes:
    - ReadWriteMany
  nfs:
    server: nfs-server.default.svc.cluster.local
    path: "/"
  mountOptions:
    - nfsvers=4.2
```

PersistentVolumeClaim

Un **PersistentVolumeClaim (PVC)** solicita almacenamiento indicando tamaño, modos de acceso y, opcionalmente, clase.

El **Pod referencia el PVC**, no necesita conocer los detalles del backend. Kubernetes vincula el claim con un PV compatible.

La solicitud puede resolverse mediante:

- **Aprovisionamiento estático:** un PV creado previamente por el administrador.
- **Aprovisionamiento dinámico:** creación de un PV mediante el provisioner de una StorageClass.

PersistentVolumeClaim • YAML estático

Este ejemplo elige explícitamente el PV `nfs` y solicita almacenamiento **sin clase**.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs
spec:
  accessModes:
    - ReadWriteMany
  storageClassName: ""
  resources:
    requests:
      storage: 1Mi
  volumeName: nfs
```

StorageClass

Una **StorageClass** describe una clase de almacenamiento disponible: rendimiento, backend o políticas. No se limita a la nube.

- **provisioner:** componente que aprovisiona los PV.
- **parameters:** opciones específicas del proveedor.
- **reclaimPolicy:** retención del almacenamiento, `Delete` o `Retain`.
- **volumeBindingMode:** `Immediate` o `WaitForFirstConsumer`.

Con **WaitForFirstConsumer**, la vinculación/aprovisionamiento espera un Pod consumidor y considera sus restricciones de scheduling y la topología del volumen.

El PVC solicita la clase con **storageClassName**; si omite el campo, puede utilizarse la clase predeterminada.

StorageClass • Aprovisionamiento dinámico

Un **driver CSI** implementa la interfaz estándar de almacenamiento sin modificar Kubernetes.

El provisioner puede crear un PV para el PVC que solicita esta clase, si no existe uno compatible.

El driver se instala aparte. Su nombre y parámetros en este ejemplo son **ilustrativos**: deben ajustarse al driver elegido.

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast
provisioner: csi-driver.example-vendor.example
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
parameters:
  type: ssd
```

PersistentVolumeClaim • Dinámico

El PVC pide la clase **fast**, sin fijar un `volumeName`.

Con `WaitForFirstConsumer`, el provisioner coordina el volumen con la ubicación del Pod consumidor.

Una vez vinculado, el Pod consume el PVC igual que en el ejemplo estático.

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: fast
  resources:
    requests:
      storage: 5Gi
```

StorageClass + PVC + PV

PVC solicita clase → provisioner crea PV

Pod → monta PVC ↔ vinculado a PV

- La **StorageClass** define cómo aprovisionar.
- El **provisioner** crea el PV; Kubernetes lo vincula al PVC.
- El **Pod** monta el almacenamiento referenciando el PVC.
- En modo estático, el administrador crea el PV; este también puede pertenecer a una clase.

Operators

Un **Operator** combina recursos personalizados con un controlador para automatizar tareas específicas de una aplicación.

- El **CRD** registra el tipo de recurso personalizado.
- El recurso declara el **estado deseado**.
- El controlador observa el **estado actual** y ejecuta un **ciclo de reconciliación** para acercarlos.
- No se limita a aplicaciones stateful ni a cambios de Deployment.
- OperatorHub.io es un catálogo; el mantenimiento depende de cada proyecto.

La reconciliación puede coordinar recursos de cómputo, red y almacenamiento.

Operators • Ejemplo PostgreSQL

Recurso **ilustrativo**, definido por el CRD de un operador:

```yaml
apiVersion: postgres.example.com/v1
kind: PostgresCluster
metadata:
  name: pagos
spec:
  version: "16"
  instances: 3
```

No pertenece a la API nativa de Kubernetes.

Al cambiar `instances` de **3 a 5**, el controlador:

- Crea las nuevas instancias.
- Ajusta replicación y configuración.
- Informa el avance mediante `status`.
- Repite el ciclo para converger al estado deseado.

La lógica específica de PostgreSQL vive en el controlador, no en el manifiesto del usuario.

ConfigMap

Para configurar una aplicación sin reconstruir su imagen, un ConfigMap almacena datos no confidenciales en pares clave-valor.

Permite desacoplar la configuración específica del entorno de las imágenes de su contenedor, para que sus aplicaciones sean fácilmente portátiles.

No proporciona confidencialidad ni encriptación. Si los datos que desea almacenar son confidenciales, use un Secreto en lugar de un ConfigMap, o use herramientas adicionales (de terceros) para mantener la privacidad de sus datos.

1MiB Max

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  # property-like keys; each key maps to a simple value
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"

  # file-like keys
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

ConfigMaps • Consumo

Los pods pueden consumir ConfigMaps y usarlos como variables de entorno, argumentos de línea de comandos o como archivos de configuración en un volumen.

Los ConfigMaps se usan dentro del Pod de la siguiente manera:

- Dentro de command (o argumentos de este)
- Como variables de ambiente para un container
- Archivos read-only montados dentro del Pod
- Consumidos por código dentro de la aplicación, por un cliente de la API de Kubernetes

ConfigMaps • Variables de entorno (ConfigMap)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"
```

ConfigMaps • Variables de entorno (Pod)

```yaml
spec:
  containers:
  - name: demo
    image: alpine
    command: ["sleep", "3600"]
    env:
    # Define the environment variable
    - name: PLAYER_INITIAL_LIVES
      # Notice that the case is different here
      # from the key name in the ConfigMap.
      valueFrom:
        configMapKeyRef:
          name: game-demo  # The ConfigMap this value comes from.
          key: player_initial_lives  # The key to fetch.
    - name: UI_PROPERTIES_FILE_NAME
      valueFrom:
        configMapKeyRef:
          name: game-demo
          key: ui_properties_file_name
```

ConfigMaps • Archivos montados (ConfigMap)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: game-demo
data:
  player_initial_lives: "3"
  ui_properties_file_name: "user-interface.properties"
  game.properties: |
    enemy.types=aliens,monsters
    player.maximum-lives=5
  user-interface.properties: |
    color.good=purple
    color.bad=yellow
    allow.textmode=true
```

ConfigMaps • Archivos montados (Pod)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-demo-pod
spec:
  containers:
  - name: demo
    image: alpine
    command: ["sleep", "3600"]
    volumeMounts:
    - name: config
      mountPath: "/config"
      readOnly: true
```

ConfigMaps • Archivos montados (volumen)

```yaml
volumes:
# You set volumes at the Pod level, then mount them into containers inside that Pod
- name: config
  configMap:
    # Provide the name of the ConfigMap you want to mount.
    name: game-demo
    # An array of keys from the ConfigMap to create as files
    items:
    - key: "game.properties"
      path: "game.properties"
    - key: "user-interface.properties"
      path: "user-interface.properties"
```

Cambios en los ConfigMaps

El mecanismo de consumo determina cómo se aplica una actualización:

- **Variables de entorno y argumentos derivados:** no cambian automáticamente; hay que recrear el Pod.
- **Volumen proyectado:** los archivos se actualizan de forma eventual mediante el kubelet.
- **subPath:** el montaje no recibe actualizaciones del ConfigMap.
- **Aplicación:** debe releer o recargar el archivo; actualizarlo no reinicia el proceso.
- **Cliente de la API:** puede leer u observar cambios sin depender del kubelet.

La demora del volumen depende de la sincronización y de la estrategia de detección de cambios del kubelet.

Secrets

Un **Secret** contiene datos confidenciales, como contraseñas, tokens o claves, separados de la imagen y del código de la aplicación.

Se crea independientemente del Pod y puede consumirse como archivo o variable de entorno.

Es similar a un ConfigMap, pero su tratamiento de acceso es diferente. **Usar un Secret no garantiza por sí solo la confidencialidad:** hay que proteger acceso, almacenamiento y consumo.

Secrets • Acceso

El acceso a la API no basta: leer o modificar Secrets requiere **autorización**, habitualmente mediante RBAC.

- **get**, **list** y **watch** pueden revelar sus datos; aplicar mínimo privilegio.
- Acceder a **etcd** evita los controles de la API y es altamente privilegiado.
- Crear Pods o Deployments puede permitir montar Secrets del mismo namespace, salvo controles adicionales.

El namespace delimita referencias, pero no reemplaza una política de seguridad.

Secrets • Almacenamiento y cifrado

En una instalación estándar, los Secrets se almacenan **sin cifrar en etcd**, salvo que se configure cifrado at-rest.

Base64 no es cifrado: codificar `data` no protege su contenido.

El cifrado se configura en el API server mediante **EncryptionConfiguration**, con claves o un proveedor **KMS**. En servicios gestionados, verificar las garantías y opciones concretas del proveedor.

Secrets • Creación

Se crean mediante `kubectl` o manifiestos. En **data**, los valores se codifican en Base64:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
data:
  username: YWRtaW4=
  password: MWYyZDFhMmU2N2Rm
```

Valores educativos, no credenciales reales.

stringData acepta texto plano; Kubernetes lo incorpora a `data` al persistir el objeto:

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: mysecret
type: Opaque
stringData:
  config.yaml: |
    apiUrl: "https://my.api.com/api/v1"
    username: <user>
    password: <password>
```

No guardar manifiestos con credenciales reales en el repositorio.

Buenas Prácticas con los Secrets

- Habilitar **cifrado at-rest** y proteger las claves de cifrado.
- Aplicar **RBAC con mínimo privilegio** y controlar la creación de workloads.
- Limitar el acceso a los contenedores que lo necesitan.
- **Rotar credenciales** y asegurar que la aplicación recargue sus valores.
- Evaluar proveedores externos según sus garantías de persistencia y actualización.

External Secrets Operator sincroniza a Secrets de Kubernetes (etcd). **Secrets Store CSI Driver** puede montar desde un proveedor externo sin sincronizarlos a Secrets.

Fin!!